

Spatial Database Management

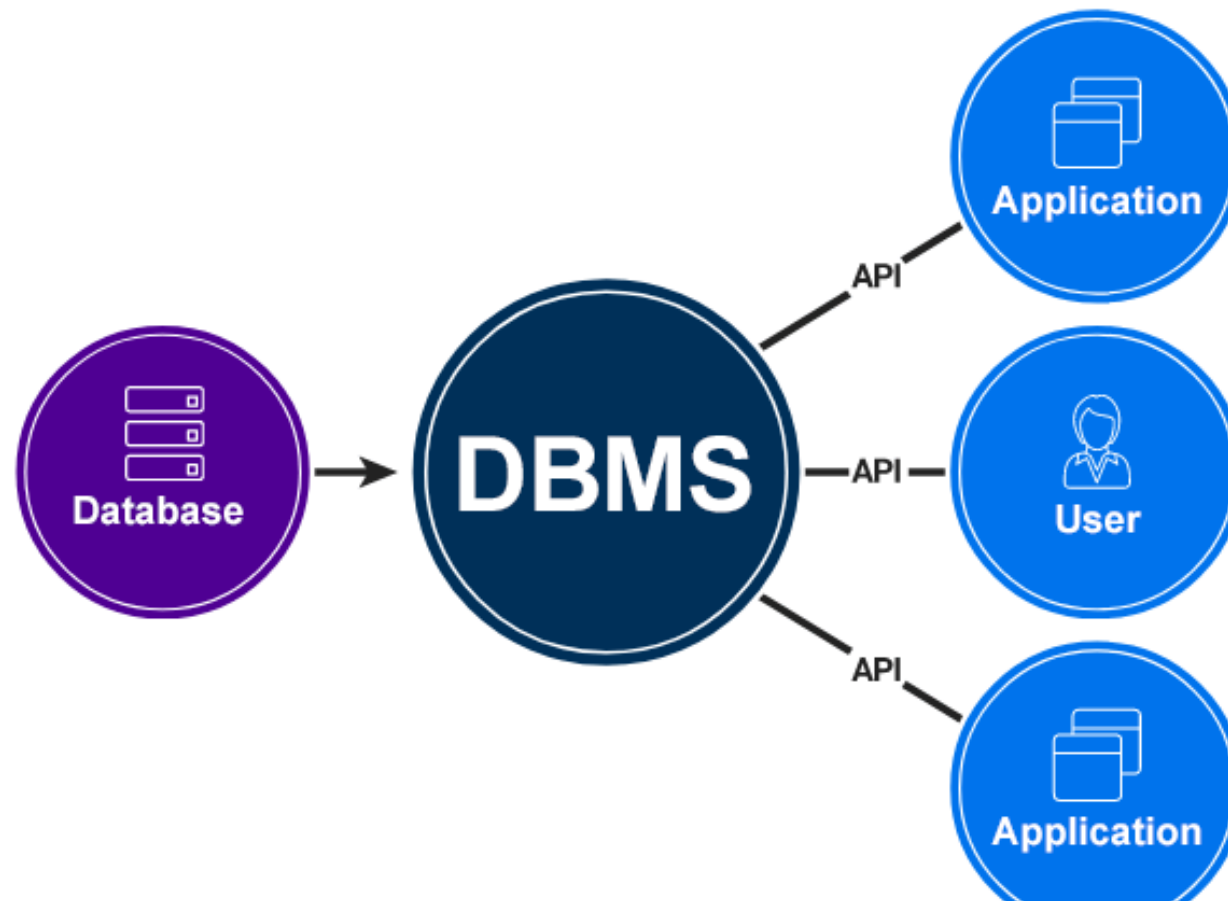
By

Dr. Meenakshi Devi

EES, Department, IISER Mohali

Introduction

- **A database is an integrated set of data on a particular subject.**
- Databases can be physically stored in files or in specialist software programs called database management systems (DBMS).



Database management systems

A DBMS is a software application designed to organize the efficient and effective storage and access of data.

Capabilities:

- **Data Model**
 - Represents real-world objects digitally.
- **Data Loading**
 - Tools to import standard data formats.
 - Custom programs can load non-standard data.
- **Query Language (SQL)**
 - Standard language for querying and manipulating data.

Capabilities....

- **Security**
 - Controls user access.
 - Different levels of access (read-only, full access) can be assigned.
- **Backup and Recovery**
 - Tools to protect data from loss or corruption.
 - Restore database in case of failure.
- **APIs (Application Programming Interfaces)**
- Allow developers to build custom applications using programming languages.

Advantages of DBMS

- Assembling all data at a single location reduces redundancy.
- Maintenance costs decrease because of better organization and reduced data duplication.
- Applications become data independent so that multiple applications can use the same data and can evolve separately over time.
- Security and standards for data and data access can be established and enforced.
- DBMS are better suited to managing large numbers of concurrent users working with vast amounts of data.

Disadvantages of DBMS

- The cost of acquiring and maintaining DBMS software can be quite high.
- A DBMS adds complexity to the problem of managing data, especially in small projects.
- Single user performance will often be better for files, especially for more complex data types and structures where specialist indexes and access algorithms can be implemented.

Types of DBMS in GIS

- **1. Relational DBMS (RDBMS)**
- Stores data in **tables (rows and columns)**.
- Each table holds attributes of real-world objects.
- **Most common** type; over **95%** of DBMS data stored in RDBMS.
- Examples: **MySQL, PostgreSQL, Oracle (basic)**.
- **Limitations:**
 - Cannot directly store **complex objects** (like geographic data).
 - Poor performance for **geographic queries**.
 - Originally designed for business applications.

2. Object-Oriented DBMS (ODBMS)

- Stores **complete objects** (both **data** and **behavior**).
- Supports **rich data types** like spatial features, sound, and video.
- Offers **object-oriented query tools**.
- Examples:
 - **GemStone/S Object Server**
 - **Objectivity/DB**
 - **ObjectStore**
 - **Versant**
- **Limitations:**
 - Technically strong but **not widely adopted**.
 - Failed to replace RDBMS due to RDBMS dominance.

3. Object-Relational DBMS (ORDBMS)

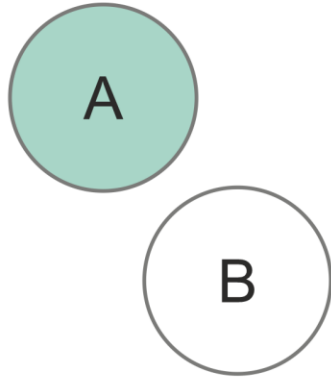
- **Hybrid** system combining RDBMS and ODBMS features.
- Adds **object support** to traditional relational systems.
- Can store both:
 - **Object attributes** (e.g., color, size),
 - **Object behaviors** (e.g., drawing, querying, spatial functions).
- Widely used in **GIS applications**.
- Examples:
 - **IBM DB2**
 - **Oracle**
 - **Microsoft SQL Server**
 - **Informix Dynamic Server**

Spatial Relationships in Databases

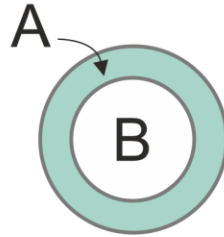
Common Spatial Relationship Operators

These are the **Boolean operators** used to test spatial relationships:

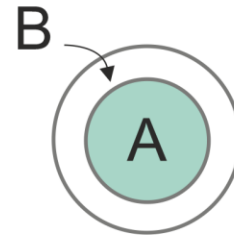
- Equals – are the geometries the same?
- Disjoint – do the geometries share a common point?
- Intersects – do the geometries intersect?
- Touches – do the geometries intersect at their boundaries?
- Crosses – do the geometries overlap (can be geometries of different dimensions, for example, lines and polygons)?



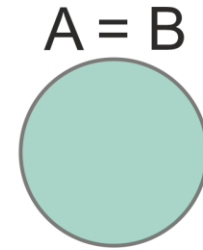
disjoint
"A and B are fully disconnected"



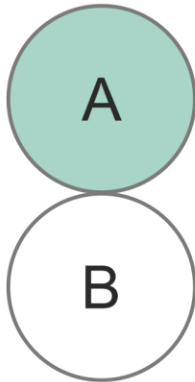
contains
"A contains B"



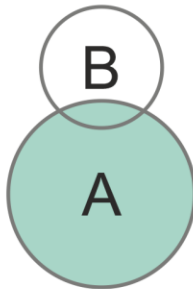
within
"A is within B"



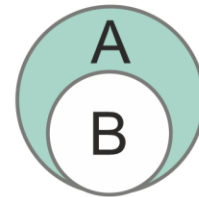
A = B
equals
"A and B are topologically equal"



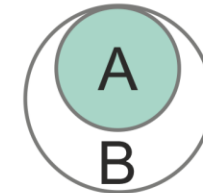
touches
"A and B touch at border"



overlaps
"A and B overlap each other"



covers
"A covers B"



covered by
"A is covered by B"

Geographic database design

1. Core Data Models

- **GIS and DBMS packages** have their own **core data models**.
- These models define:
 - **Object types** (e.g., points, lines, polygons).
 - **Relationships** between objects.

2. DBMS Data Models

- DBMS define models for:
 - **Data types** (e.g., geometry types).
 - **Access functions** (e.g., SQL-based queries).
- Support for **simple features**: points, lines, polygons.

3. GIS Extended Models

- GIS builds on DBMS to include advanced features like:
 - **TINs (Triangulated Irregular Networks)**
 - **Topologies**
 - **Feature-linked annotations**
- Enables representation of **complex spatial relationships**.

4. Domain-Specific Data Models

- GIS models can be extended with **domain data models** tailored to specific industries.
- Examples:
 - **Water utilities**
 - **Parcel maps**
 - **Census geographies**

5. Project-Specific Data Models

- Individual projects create **physical data model instances** based on templates.
- Example:
 - A **City Planning Department** builds a **sewer line database** using a **water/wastewater model** layered on GIS and DBMS models.

Levels of Data Modeling

There are **three stages** of data modeling:

- **Conceptual model** – defines what data is needed at a high level.
- **Logical model** – defines how data is structured in the system.
- **Physical model** – defines how data is stored and accessed in the actual database.

Levels of Data Modeling

1. Conceptual Model

Purpose: Models the **user's view** of the system.

- Examples: managing forestry resources, locating vacant land, maintaining highways.
- **Determine data needs**
 - What data is needed to support each function?
- **Organize data into groups**
 - For efficient data management.
- **Define objects and their relationships**
 - Specify object types (classes) and how they relate to each other.
 - Use object modeling techniques to visualize relationships.
- **Select geographic representation types**
 - Choose between:
 - **Discrete objects** (points, lines, polygons)
 - **Fields** (continuous surfaces)

2. Logical Model

Purpose: Translates the conceptual model into a **GIS-compatible structure**.

- **Match object types to geographic data types**
 - Align conceptual objects with the data types supported by the target GIS system.
- **Independent of storage platform**
 - Logical model is not tied to any specific DBMS (e.g., can work with Oracle, Access, or file systems).
- **Organize database structure**
 - Define:
 - **Topological relationships** (e.g., adjacency, connectivity)
 - **Rules and associations** between objects
 - **Coordinate system and projection**

3. Physical Model

Purpose: Builds the **actual database** structure that stores data.

- **Define the database schema**
 - Specifies how data is physically stored in the database.
- **Use data definition language (DDL)**
 - Most commonly **SQL with geographic extensions** (e.g., PostGIS, Oracle Spatial).
- **Adapt to DBMS capabilities**
 - SQL variants may differ across systems, especially in older GIS setups.